$$\underbrace{ST_n}_{(D_n, D'_n)} = \begin{cases} 1 & \text{if } (D_n + D'_n) \geq 10 \\ 0 & \text{if } (D_n + D'_n) \leq 8 \text{ or } n = 0 \\ U & \text{otherwise} \end{cases} \quad \begin{array}{l} \text{Note ST0 is always 0 or 1.} \\ \text{Sums to 9, so carry is uncertain} \end{array} \tag{2}$$

The "TriAdd" function combines two ST_n style values X and Y. Only if *both* X and Y are U, does TriAdd return U (there is still uncertainty), otherwise it returns 0 or 1 (the carry bit value is known):

$$\underbrace{\text{TriAdd}}_{(X,Y)} = \begin{cases} Y & \text{if } X = U \\ X & \text{otherwise} \end{cases} \tag{3}$$

Finally, we introduce the SV subtask that uses TriAdd to handle multidigit carry cascades. We describe SV_n using the first 3 examples:

$$SV0 = ST0 \tag{4}$$
$$SV1 = \text{TriAdd}(ST1, ST0) \tag{5}$$
$$SV2 = \text{TriAdd}(\text{TriAdd}(ST2, ST1), ST0) \tag{6}$$

The highlighted (solid-color) part of Figure 1 illustrates the SV2 calculation. Note that the calculation first combines ST2 and ST1 (the higher value digits) before combining the result with ST0. This formulation mirrors the left to right order in which the model processes tokens i.e. from higher value digits to lower value digits. The last term in all SV_n calculations is ST0 which is always 0 or 1, so all SV_n evaluate 0 or 1. That is, at the end of the SV_n calculation any uncertainty has been resolved.

The SV_n are perfectly accurate multidigit cascade carry values. Their calculation occurs across multiple tokens and is completed by the "=" token. Combining the SV_n values with the SA_n values as shown in Figure 1 gives perfectly accurate answer tokens.

3.3 Subtraction Algorithm Description

Our subtraction algorithm mirrors our addition algorithm, replacing addition's "cascading carry one" with subtraction's "cascading borrow one". Both operations use the same iterative approach to reducing "cascading" uncertainty token by token.

We introduce "Base Diff" MD_n defined as D_n- D'_n modulo 10 (paralleling SA_n). We also introduce MB_n for the single-digit "borrow one" case (paralleling ST_n):

$$\underbrace{MB_n}_{(D_n, D'_n)} = \begin{cases} 1 & \text{if } D_n < D'_n \text{ (borrow one)} \\ 0 & \text{if } D_n > D'_n \text{ (no borrow)} \\ U & \text{if } D_n = D'_n \text{ (uncertain)} \end{cases} \tag{7}$$

Finally, we introduce MV_n which parallels SV_n to handle the "cascading borrow one" edge case. With this formulation, the addition and subtraction algorithms have the same structure (just replacing SA_n with MD_n, ST_n with MB_N, and SV_n with MV_N).

Subtraction poses an additional algorithmic difficulty - some questions give a positive answer (e.g. 325-129=+196) and others a negative answer (e.g. 325-329=-004). Note that all the answer digits are different in the above examples. While there are multiple ways this could be handled mathematicaly, we introduce "Neg Diff" ND_n defined as D'_n- D_n modulo 10 (the opposite of MD_n). At the "=" token, the MV_n avalues are known, and the algorithm selects MD_n or MD_n values as appropriate.

In Fig. 4, the value $MV2$ determines the answer sign. While the following calculations could use $MV2$ too, in fact they attend to the answer sign itself. We introduce the subtask SGN to reflect this.

3.4 Mixed Algorithm Description

For mixed models handling both operations, we add the subtask OPR meaning "attends to the prompt operation token". The mixed models uses OPR and SGN to determine whether to output addition, positive-subtraction, or negative-subtraction answer digits.

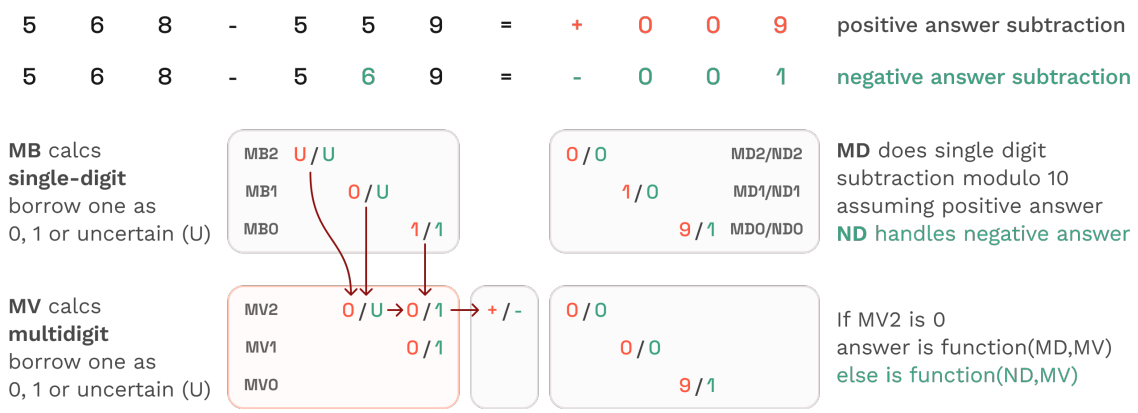

Figure 4: Our subtraction algorithm parallels our addition algorithm but it refines "cascading borrow one" uncertainty over multiple tokens. The refined $MV2$ value determines both the answer sign ("+" or "-") and which digit values (MD_n or ND_n) to use for the final output.

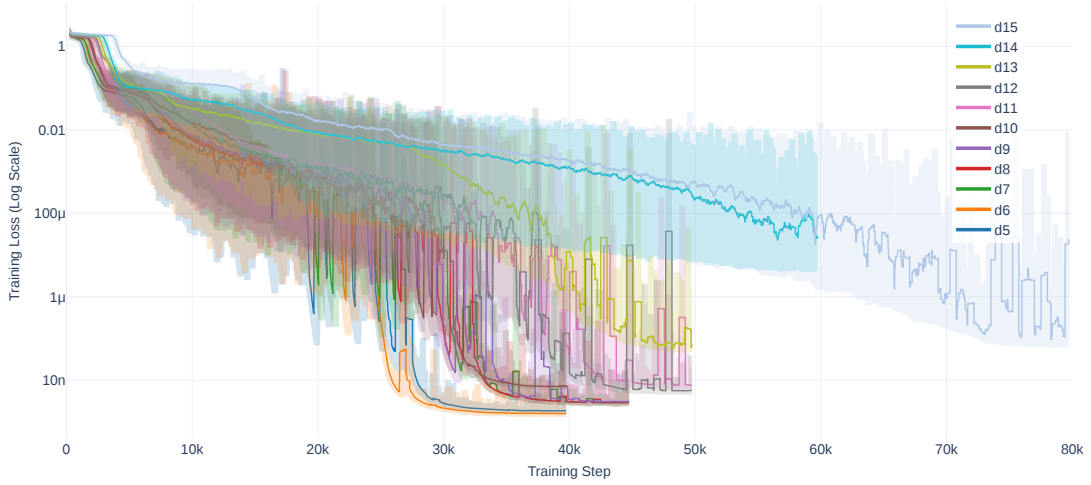

Figure 5: The 5- to 15-digit 2-layer 3-head addition models have very low loss. With more digits training takes longer. Details in Tab.5.

4 Experiments

4.1 Training Models

Our models' vocabulary consisted of 14 tokens: the digits 0-9 plus the mathematical operators +, -, =, *, and /. (Also see App. C). We created an infinite training dataset enriched with rare numerical combinations (mainly cascading use cases). (Also see App. D)

We trained addition-only, subtraction-only, and mixed (addition and subtraction) models. We systematically explored different model architectures and settled on 2 layers and 3 or 4 attention heads. (Also see App. F)

We trained used a batch size of 64, learning rate of 0.00008, and weight decay of 0.1. The loss function was defined as the mean negative log likelihood across all output tokens. Training was stopped when the loss was very low e.g. 2×10^{-8}. (Also see App. G, Fig. 5, Fig. 9).

Each model's accuracy was tested on 1 million questions. Robust statistical accuracy ranges were calculated, but for ease of presentation, we say a model has 99.999% accuracy if it got less than 10 of the 1 million questions wrong. Most models achieved this. The resulting 49 models, including detailed accuracy results, are summarized in Tab. 5 and Tab. 6.

4.1.1 Training mixed models

To explore how transformers can learn multiple arithmetic operations simultaneously, we conducted experiments with models capable of both addition and subtraction. We were particularly interested in whether knowledge from addition-specialized models could transfer to this expanded task.

We trained 14 mixed models with various architectures, including 6- and 10-digit models with 2-3 layers and 3-4 attention heads (see Tab.6). Each model was initialized using weights from a highly accurate addition-only model, specifically copying the weights into the first two layers and first three heads of the mixed model. During training, we used a curriculum of 80% subtraction and 20% addition examples, using enriched data for both operations (See Fig. 6).

This initialization approach proved effective, with most models quickly achieving high accuracy on both operations. We explored alternative training strategies, such as periodically "freezing" the inserted attention heads or MLPs by restoring their original addition-model weights every 100 training steps. However, these more constrained approaches resulted in lower accuracy, suggesting that some flexibility in modifying the initialized weights was necessary for optimal performance (Also see App. K)

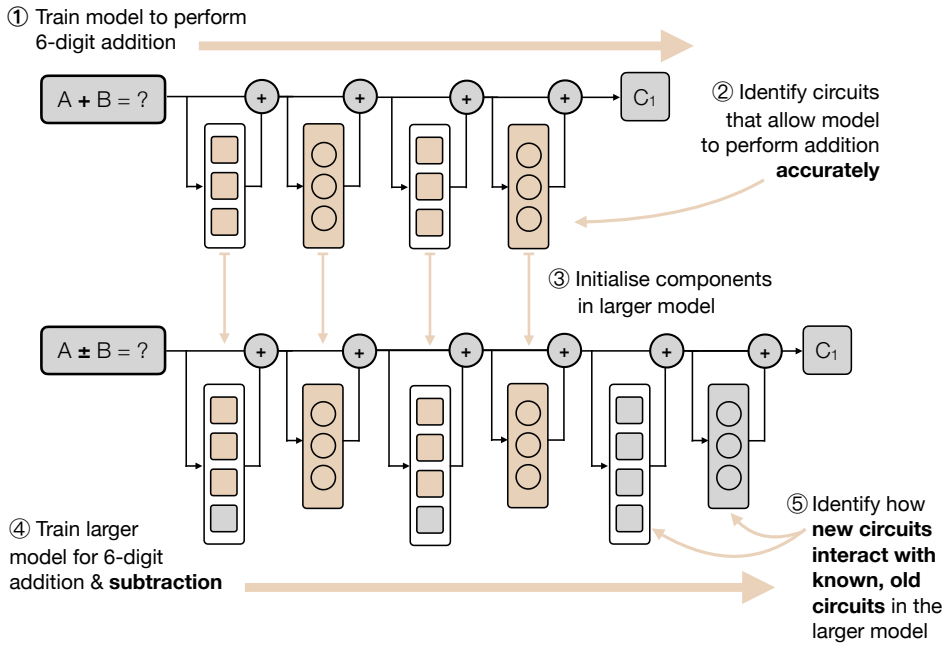

Figure 6: Mixed model initialization and analysis: (1) Train an accurate 5-digit addition model. (2) Reverse-engineer it to identify the implemented algorithm. (3) Insert its attention head and MLP weights (in brown) into a larger model. (4) Train the new model on 80% subtraction and 20% addition. (5) The resulting model predicts accurately and reuses the inserted addition circuits for both tasks.

We did a seed sensitivity analysis. App. L includes a visualization Fig. 15 of the range of training losses for different model categories.

4.2 Experimental Results

To verify whether our models implement the algorithms described above, we developed a systematic framework for analyzing model internals. For each trained model, we investigated whether it contains the specific computational components our algorithms require and whether these components interact according to the algorithmic constraints.

4.2.1 Implementation Analysis Framework

Consider a 6-digit addition model. If it implements our algorithm, it must satisfy these constraints:

- **Functional accuracy**: The model must correctly add any two 6-digit numbers, selecting the unique correct answer from nearly 2 million possible answers.